

Detailed Project Report

TechNewsAggregator

CISC3016 Project: Creating a Web Page from Other Web Sites

1. Introduction

This project, TechNewsAggregator, was developed for the course requirement "Creating a web page from other web sites." The main objective of the assignment is to gather live information from the Internet and create new web content around a chosen theme. In this project, the selected theme is technology news aggregation.

The final system collects live news data from public technology news websites, stores the fetched results locally, and presents them through a dynamic web page. In addition to basic aggregation, the project also implements image interaction, sound playback, random relevant comments, backend-based spell checking, and responsible web scraping practices. The development and repair process was assisted by a large language model, and the final system includes a comparative analysis between hand-built development and LLM-assisted development.

2. Project Objectives

The main objectives of this project are:

- To collect live information from at least three external websites.
- To present the collected information on a themed web page.
- To enrich the page with pictures, sound links, and random relevant comments.
- To support image zoom on hover or click.
- To implement spell checking through the website backend using an online dictionary.
- To demonstrate code generation and repair with the help of a large language model.
- To follow responsible web scraping practices throughout the crawler implementation.

3. Theme Selection

The selected theme is **technology news**. This theme was chosen because:

- Technology news websites frequently update their public feeds, making them suitable for live information gathering.
- The topic is broad enough to provide varied content while still remaining coherent.
- It allows the project to demonstrate aggregation behavior similar to sites such as Google News.

- It fits well with additional features such as images, sound, comments, and comparative discussion.

4. Data Sources

The project currently uses the following three public technology news sources:

- TechCrunch
- The Verge
- Engadget

These websites provide publicly accessible RSS or Atom feeds. They were selected because they are relevant to the project theme, provide live content, and can be accessed in a responsible and compliant manner under the current implementation.

The source configuration is implemented in [scraper.py](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/scraper.py).

5. Requirement-by-Requirement Mapping

5.1 Create a new class called `Web`

The project includes a dedicated Web class in [Web.py](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/Web.py). This class encapsulates:

- HTTP request handling
- HTML and feed parsing
- RSS and Atom extraction logic
- request throttling
- `robots.txt` checking
- helper methods for content cleaning and feed processing

This satisfies the assignment requirement to implement the new methods in a Web class.

5.2 Create a web page using information from at least three other websites

The project collects live information from three external websites and stores the results in [scraped_news.json](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/scraped_news.json). The data is then displayed on a web page through a Flask backend and dynamic client-side rendering.

This satisfies the requirement of using at least three other websites.

5.3 Add pictures, a sound link, and random relevant comments

The page includes:

- pictures for each article card

- audio links that play local `.wav` sound files
- random relevant comments based on source or topic category

The frontend logic is implemented in [index.html](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/index.html), and audio resources are stored in the assets/audio/ directory.

5.4 Zoom in on the image when clicked or hovered

The image behavior includes:

- scaling on mouse hover
- a zoom overlay when an image is clicked

This satisfies the image zoom interaction requirement.

5.5 Spell checking through the website backend

The spell checker is implemented through the local Flask backend in [app.py](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/app.py). The frontend sends the word to POST /api/spell-check, and the backend accesses an online dictionary API to verify spelling and return results.

This design explicitly satisfies the requirement that the online dictionary must be integrated through the website backend, not directly from browser-side JavaScript.

5.6 Use automatic code generation with a large language model and provide comparative analysis

The project development process used a large language model as an implementation and repair assistant. The final project includes comparative discussion on the strengths and weaknesses of:

- building a website from scratch
- generating or accelerating development with a large language model

This requirement is addressed in both the webpage content and the project report.

6. System Architecture

The final project follows a simple three-layer structure:

6.1 Crawling Layer

The crawling layer is responsible for retrieving live content from public news feeds. It is implemented through:

- [Web.py](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/Web.py)
- [scraper.py](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/scraper.py)

This layer handles:

- requests to the feed sources
- RSS and Atom parsing
- feed item normalization
- robots checking
- delay and rate limiting

6.2 Backend Layer

The backend layer is implemented using Flask in [app.py](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/app.py). It provides:

- ``/`` for serving the main page
- ``/api/news`` for returning collected news data
- ``/api/spell-check`` for spell checking
- ``/api/health`` for status verification

This layer acts as the bridge between data storage and the dynamic webpage.

6.3 Frontend Layer

The frontend is implemented in [index.html](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/index.html). It is responsible for:

- requesting news from the backend
- dynamically generating article cards
- showing images and comments
- playing sound files
- handling zoom interactions
- performing spell-check requests through the backend

7. Implementation Details

7.1 Feed Collection and Parsing

The crawler originally relied on a simpler feed parsing strategy, which worked for standard RSS feeds but failed for Atom feeds such as The Verge. To solve this, the parser was upgraded to support both RSS and Atom using XML parsing.

Each collected item includes fields such as:

- ``title``
- ``link``
- ``description``
- ``pubDate``
- ``source``

- `category`
- `ai\_comment`

This change significantly improved data completeness and reliability.

7.2 Dynamic News Rendering

The initial version of the webpage used hard-coded article cards, which did not satisfy the requirement of displaying live Internet information. This was repaired by introducing a backend API and a frontend rendering flow:

1. run the scraper to generate `scraped_news.json`
2. start the Flask backend
3. load `/api/news` from the webpage
4. render the returned items into article cards

This makes the site data-driven rather than static.

7.3 Spell Checking Workflow

The final spell-checking workflow is:

1. the user enters a word on the webpage
2. the frontend sends the word to `/api/spell-check`
3. the backend calls an online dictionary API
4. the backend returns correctness, part of speech, and definition
5. the webpage displays the result

This architecture matches the assignment wording and separates frontend presentation from external service access.

7.4 Audio Feature

The assignment requires a sound file link that plays when the user clicks it. The repaired project uses local `.wav` files stored under `assets/audio/`. Different news sources map to different audio files. This is more appropriate than using placeholder links or only generating synthetic beeps in JavaScript.

7.5 Random Relevant Comments

The page includes random comments that are selected according to the news source or topic category. This makes the content feel more dynamic and better aligned with the live theme, even though the comments are template-based rather than generated by a separate runtime AI service.

7.6 Image Zoom Interaction

The site supports:

- hover-based image enlargement
- click-to-zoom overlay
- keyboard escape to close the enlarged image

This improves the user experience and directly matches the assignment requirement.

8. Responsible Web Scraping

Responsible scraping was a required part of the project. The crawler was implemented to follow the required principles:

- Respect `robots.txt`
- Do not bypass CAPTCHA or authentication systems
- Only access public data
- Use delays and request throttling
- Avoid harmful or exploitative behavior

The `robots.txt` logic is implemented directly in the request path in `[Web.py]`(`file:///D:/■■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/Web.py`), rather than only existing as an unused helper. This means compliance is not merely described in comments; it is part of actual runtime behavior.

Additional notes are documented in `[scraping-compliance.md]`(`file:///D:/■■■■■/university/2026/CISC3016/proj/docs/scraping-compliance.md`).

9. Output Results

The assignment asks for both code and output results. In this project:

- source code is stored in the `TechNewsAggregator` project directory
- the output data is stored in `[scraped_news.json]`(`file:///D:/■■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/scraped_news.json`)
- the output file contains the latest fetched news entries from the three selected websites

At the current stage, the output contains 15 articles in total, with 5 items per source under the latest successful run.

10. Testing and Validation

The project was tested through both functional checks and manual review.

10.1 Functional Validation

The following checks were performed:

- ``python scraper.py`` runs successfully
- ``python app.py`` starts the local web server
- ``GET /api/news`` returns valid JSON data
- ``POST /api/spell-check`` returns dictionary validation results
- local audio files can be accessed successfully
- image zoom interactions work

10.2 Manual Acceptance Review

A manual acceptance flow was carried out to verify:

- the webpage loads correctly
- multiple article cards are displayed
- the three selected sources appear on the page
- article links are usable
- spell checking works for correct and incorrect inputs
- image zoom and sound playback behave as expected
- the page does not show obvious blocking errors during normal use

The manual review did not reveal any major issues that would prevent submission.

11. LLM-Assisted Development and Comparative Analysis

A comparative analysis is presented below to evaluate the advantages and disadvantages of building a website from scratch versus generating it with a large language model.

11.1 Advantages of Building with LLM Assistance

- Faster prototyping of frontend and backend components
- Quicker generation of boilerplate code
- Efficient support during repair and iteration
- Helpful for restructuring code to match assignment requirements

11.2 Disadvantages of LLM Assistance

- Initial outputs may look complete while still missing required runtime integration
- Generated code may not fully understand real feed formats or compliance constraints
- Human review is still required to validate correctness and completeness
- Debugging and final polishing still depend heavily on manual reasoning

11.3 Advantages of Building from Scratch

- Greater control over design and architecture
- Better consistency when written carefully by hand

- Easier to reason about each implementation decision from the beginning

11.4 Disadvantages of Building from Scratch

- Slower development time
- Higher implementation overhead for a short course project
- More time spent on boilerplate and routine features

11.5 Final Comparison

For this project, the most effective approach was not purely manual development and not purely generated development. Instead, the best result came from combining both:

- use LLM assistance to accelerate coding and repairs
- use human review to compare the output against the assignment
- use testing and iteration to close the gap between appearance and correctness

This combination produced a stronger final submission than either approach alone.

12. Challenges Encountered

Several practical issues were encountered during development and repair:

- the original page displayed static content rather than live data
- the original spell checker called the dictionary API directly from the browser
- RSS parsing did not support Atom feeds
- one original source was not suitable under the stricter `robots.txt` implementation
- the sound feature originally relied on placeholder behavior rather than real files

These issues were resolved through backend integration, parser updates, source replacement, audio resource generation, and documentation improvements.

13. Project Files

Important project files include:

- [Web.py](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/Web.py)
- [scraper.py](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/scraper.py)
- [app.py](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/app.py)
- [index.html](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/index.html)
- [scraped_news.json](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/scraped_news.json)

- [README.md](file:///D:/■■■■/university/2026/CISC3016/proj/project/project/TechNewsAggregator/README.md)
- [project-report.md](file:///D:/■■■■/university/2026/CISC3016/proj/docs/project-report.md)
- [work-orders.md](file:///D:/■■■■/university/2026/CISC3016/proj/docs/work-orders.md)

14. Conclusion

In conclusion, TechNewsAggregator successfully fulfills the major requirements of the assignment "Creating a web page from other web sites." The project gathers live information from three external technology news sources, renders the results dynamically on a webpage, includes pictures, sound playback, random comments, image zoom, and backend-based spell checking, and incorporates responsible web scraping practices.

The project also demonstrates how large language model assistance can accelerate development, while still requiring careful human review, testing, and refinement. As a result, the final system is not only functional, but also better aligned with the original project specification and suitable for course submission.